

Framework

The framework of this algorithm is built with focus on expandability. Currently, nine blocks are implemented. The way the blocks are stored in the global variables, new ones can be added without any major changes. In general, the design idea is, to have to change existing functionality as little as possible, when adding new one.

Blocks

In the current development state, the program handles the following blocks:

- error block
- air
- solid block
- redstone block
- redstone dust
- north-facing redstone torch
- east-facing redstone torch
- south-facing redstone torch
- west-facing redstone torch

The program works with the abbreviations saved in the global variable BLOCKS. For the decorated version of the output BLOCKS_FANCY is used instead. The last five blocks of the fancy output are state-dependent.

```
1 (define BLOCKS      ' ("xx" "ai" "so" "rb" "rs" "tn" "te" "ts" "tw"))
2 (define BLOCKS_FANCY ' ("??" " " " 0" " X" ".*" " !~" " !>" " !v" " !<"))
```

Settings

There are 13 settings, of which four affect the synthesis. Out of the other nine, two change the way the two-dimensional grids are display and seven manage how and if the commands are printed.

```
1      (define nCOLs 2)
2      (define nROWS 2)
3
4      (define USE_SPECIFIED_COSTS #f)
5      (define SPECIFIED_COSTS '(
6          99 ; error
7          0  ; air
8          1  ; solid block
9          9  ; redstone block
10         2  ; redstone dust
11         4  ; torch north
12         4  ; torch east
13         4  ; torch south
14         4)) ; torch west
15
16      (define PRINT_STEPS #f)
17      (define PRINT_LAYERS '(
18          #t ; blocks
19          #f ; initial redstone
20          #t ; initial fancy
21          #f ; step redstone
22          #t)) ; step fancy
23
24      (define PRINT_COMMAND #t)
25      (define PRINT_CLEAR_COMMAND #t)
26
27      (define xOFFSET 0)
28      (define yOFFSET 0)
29      (define zOFFSET 0)
30      (define REPLACE_COMMAND_BLOCK #t)
31      (define PRETTIFY_COMMANDS #t)
```

Synthesis Settings

There are two main challenges in regards to redstone circuits. Firstly, knowing if a desired behavior is even possible to achieve in a given space. Secondly, building the cheapest possible circuit.

The two variables `nCOLs` and `nROWS` determine the size of the world, in which the synthesis tries to find a satisfying assignment and thereby an appropriate circuit. If `USE_SPECIFIED_COSTS` is set the `true`, the values in `SPECIFIED_COSTS` are used for calculating the overall cost of the complete circuit. Enabling this feature and setting the cost of the blocks can be important, if the resources to craft some of the blocks are more limited than others.

Output Settings

The booleans in `PRINT_LAYERS` specify, which of the five layers are printed. It is also possible to not choose any, which will lead to the output only consisting of status messages, like "SAT with minimal cost bound 16". The variables in lines 20 and 22 are combination outputs of the block-layer and the redstone-layer.

If `PRINT_STEPS` is set to `true`, every step of both optimizations is printed and not only the final results.

Apart from the settings regarding the output of the layers, there are seven more, which manage the behavior of the command output. If `PRINT_COMMAND` is not activated, all the following settings can be skipped, as no command will be printed. If `PRINT_CLEAR_COMMAND` is set to `true`, a second command will be printed, which can be used for removing the circuit. It does not simply clear the whole area, specified by `nCOLs` and `nROWS`, but it replaces all blocks set by the original command with air.

The three `OFFSET` variables can be used to offset the whole circuit in any direction.

The settings `REPLACE_COMMAND_BLOCK` is responsible for spawning an empty command block at the position of the old one. Normally it would get automatically removed after the command creates the circuit. This can come in handy, if the clear command will be inserted afterwards.

The last option is to enable `PRETTIFY_COMMANDS`, which will add newlines and proper spacing, so that the command is more human-readable.

Goals

Adding goals is the main user input. A goal can specify, which block and which redstone level must be at specific coordinates. There are only two base goals, all others are combinations of the base goals and possibly already combined goals.

Set External Input

The first method, (`set_input col row name str1 str2`) is not really a goal. It overrides symbolic variables and statically sets the given value. The program will thereby not try to find a fitting value for those coordinates, as they are already set to a specific value by the user input. The location is set by `col` and `row` and the parameter `name` defines which block is to be placed. The variable name must be one of the available blocks in the global variable `BLOCKS`. As there is the initial redstone state and the secondary one, two parameters are needed for setting the strength of the external input in each state.

Base Goals

There exist two methods for adding the two base goals, `add_block_goal` and `add_str_goal`. The first one is used to define that a specific block must be placed at specific coordinates. The simplest case, of only setting one block, is done in lines six and seven. It is also possible to set multiple blocks at the same time. This can be achieved by passing a list of integers for either one or both of the variables `col` and `row`. Lines 9 to 11 handle the case of `col` being an integer and `row` being a list. Lines 13 to 15 are used for the inverse and lines 17 to 20 manage the situation of both being lists.

One examples for each of the four possible input variations of the last two parameters:

- (`add_block_goal "so" 0 0`)
- (`add_block_goal "rs" 0 '(0 1 2)`)
- (`add_block_goal "rb" '(0 1 2) 0`)
- (`add_block_goal "ai" '(0 1 2) '(0 1 2)`)

Combined Goals

Combined goals have the benefit of not having to write any parameter twice. This can also lead to the prevention of creating an error. In the scenario of having these two goals:

- (add_block_goal "rs" 1 2)
- (add_str_goal = 13 1 2)

wanting to alter the coordinates can lead to the mistake, that the change is only done for one of the goals:

- (add_block_goal "rs" 1 3)
- (add_str_goal = 13 1 2)

If the combined goal was used instead, this error could not have happened:

1. (add_block_str_goal "rs" = 13 1 2)
2. (add_block_str_goal "rs" = 13 1 3)

The first combined goals is the most trivial succession of the two base goals.

```
1 (define (add_block_str_goal name operator str col row [is_initial #t])
2   (add_block_goal name col row)
3   (add_str_goal operator str col row is_initial))
```

It can be assumed that the user is not always interested, in which direction the redstone torch is facing. The function add_torch_goal enables to ask for any of the four possible redstone torches. The function is_torch? returns true, if and only if the checked block is any of the four torches.

```
1 (define (is_torch? number)
2   (or
3     (eq? number (get_number "tn"))
4     (eq? number (get_number "te"))
5     (eq? number (get_number "ts"))
6     (eq? number (get_number "tw"))))
7
8 (define (add_torch_goal col row)
9   (assert (is_torch? (get_block col row))))
```

The next two goals are a combination of a torch-goal and either a strength-goal of zero or anything above zero. This combination is very common, as the user is mostly interested in specifically a lit or an unlit torch at a given location and not just in any torch.

```
1      (define (add_unlit_torch_goal col row [is_initial #t])
2        (add_torch_goal col row)
3        (add_str_goal = 0 col row is_initial))
4
5      (define (add_lit_torch_goal col row [is_initial #t])
6        (add_torch_goal col row)
7        (add_str_goal > 0 col row is_initial))
```

The previous two combined goals can again be combined. Often it is the case to want an unlit torch in the initial redstone state and a lit one in the secondary state, or wise versa. This can easily be achieved with the following two goals.

```
1      (define (add_off_on_torch_goal col row)
2        (add_unlit_torch_goal col row))
3        (add_lit_torch_goal col row #f)
4
5      (define (add_on_off_torch_goal col row)
6        (add_lit_torch_goal col row)
7        (add_unlit_torch_goal col row #f))
```

Finally, there exist two goals that are mainly like the previous two. The only difference is the fact that the block does not have to be a redstone torch, but it can also be a solid block.

```
1      (define (add_off_on_torch_or_solid_goal col row)
2        (assert (or
3          (is_torch? (get_block col row))
4          (eq? (get_block col row) (get_number "so"))))
5        (add_str_goal = 0 col row)
6        (add_str_goal > 0 col row #f))
7
8      (define (add_on_off_torch_or_solid_goal col row)
9        (assert (or
10          (is_torch? (get_block col row))
11          (eq? (get_block col row) (get_number "so"))))
12        (add_str_goal > 0 col row)
13        (add_str_goal = 0 col row #f))
```
